

Polinomio Interpolante de Lagrange

El polinomio interpolante de Lagrange es una técnica matemática utilizada para aproximar funciones basándose en un conjunto de puntos dados. este método construye un polinomio que pasa exactamente por esos puntos.

$$P(x) = \sum_{i=0}^n y_i L_i(x), \quad L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x-x_j}{x_i-x_j}$$

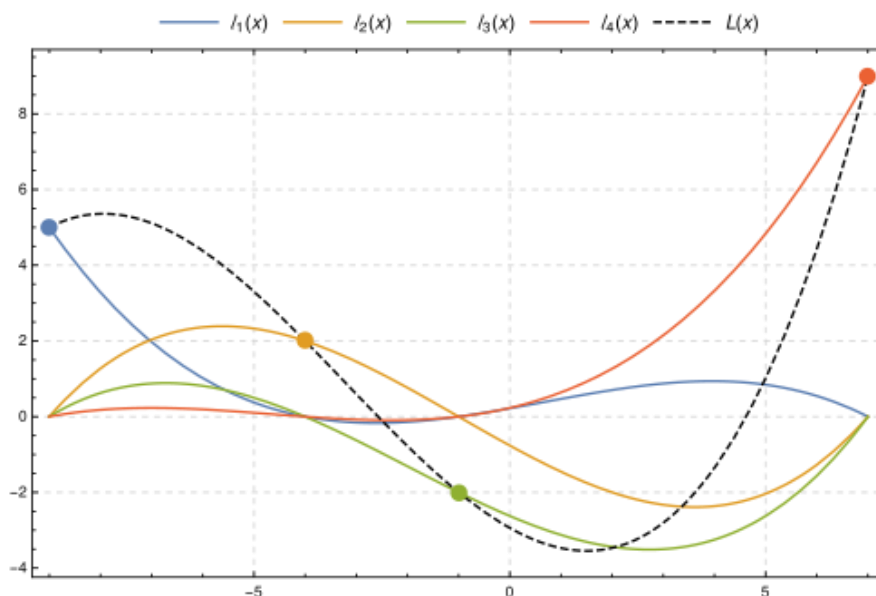


FIGURA 8

Existencia y unicidad

Siempre es posible construir un polinomio de interpolación de Lagrange para un conjunto de puntos distintos (x_i, y_i) , como $P(x)$.

$$P(x) = \sum_{i=0}^n y_i L_i(x)$$

Unicidad

Dado un conjunto $(n+1)$ puntos distintos (x_i, y_i) , existe un polinomio grado n que pasa exactamente por todos esos puntos. Esto se garantiza por el teorema fundamental de la interpolación.

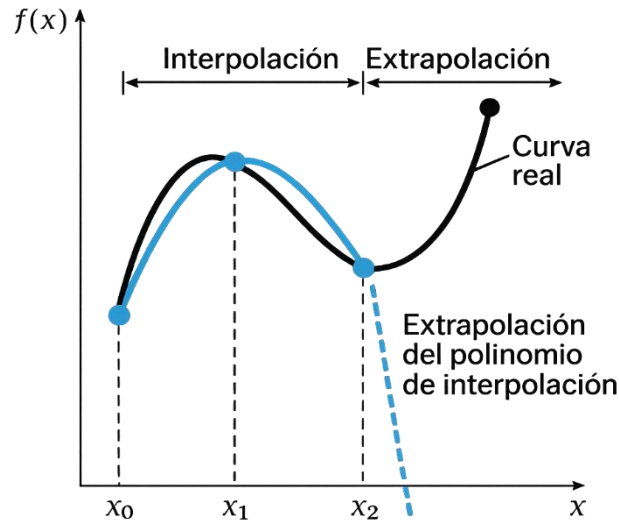


FIGURA 9

Básicamente lo que se busca es: conseguir un polinomio que según su grado sea similar a la función original que está en estudio, o en todo caso si tenemos un paquete de datos discretos construir un polinomio que permita interpolar valores en un dominio dado con cierto nivel de presión, un error local y global pequeño. En la siguiente figura vemos como parte de la función tangente se compara con un polinomio que la interpola.

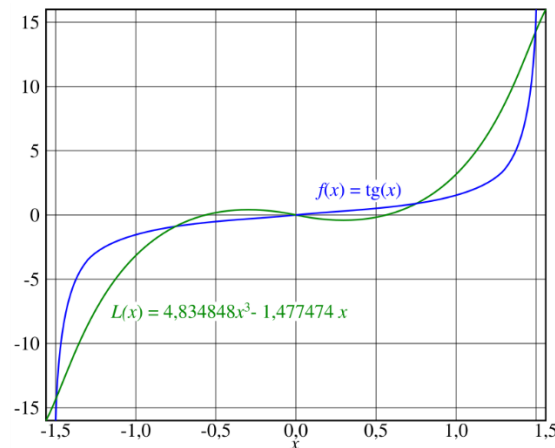


FIGURA 10

Errores locales y Globales

El error de interpolación: el error entre $f(x)$ y el polinomio $P(x)$, donde $x_0 < \xi < x_n$, es un número en el intervalo de los puntos.

$$f(x) - P(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

Aproximación de Taylor

la serie de Taylor es una herramienta matemática que permite aproximar funciones mediante una suma infinita de términos polinómicos. cada término se calcula utilizando las derivadas de la función en un punto específico.

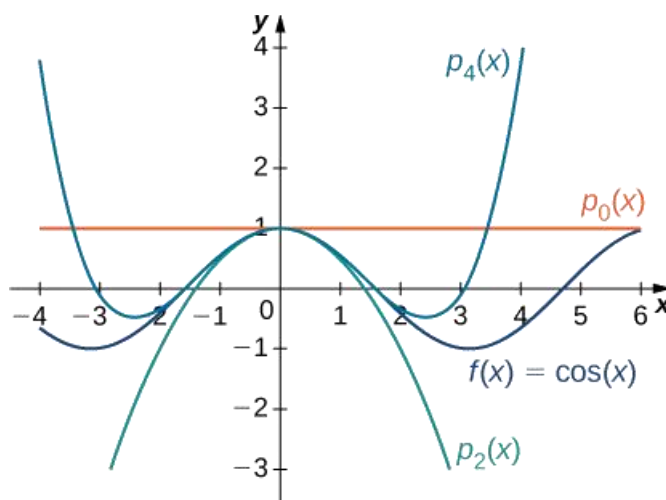


FIGURA 11

Cálculo de errores

1. Construir un polinomio interpolante de Lagrange.

$$P(x) = \sum_{i=0}^n y_i L_{i(x)}$$

2. Calcular las bases de Lagrange

$$L_{i(x)} = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

3. Determinar el valor máximo en la derivada

$$M_{n+1} = \max |f^{(n+1)}(\xi)|$$

4. Aplicar la cota

$$|E(x)| \leq \frac{M_{n+1}}{(n+1)!} \left| \prod_{i=0}^n (x - x_i) \right|$$

5. Verificar con el valor real (error local)

$$|E_{(x)}| = |f_{(x)} - P_{(x)}|$$

Código

```
import numpy as np
import matplotlib.pyplot as plt
def polinomio_lagrange(x, x_puntos, y_puntos):
    n = len(x_puntos)
    L = 0
    for i in range(n):
        # Calcular el polinomio base de Lagrange
        li = 1
        for j in range(n):
            if i != j:
                li *= (x - x_puntos[j]) / (x_puntos[i] - x_puntos[j])
        L += y_puntos[i] * li
    return L
def reconstruccion_lagrange(x_puntos, y_puntos):
    n = len(x_puntos)
    coeficientes = np.zeros(n)
    for i in range(n):
        # Calcular el polinomio base de Lagrange
        li = np.poly1d([1])
        for j in range(n):
            if i != j:
                li *= np.poly1d([1, -x_puntos[j]]) / (x_puntos[i] - x_puntos[j])
        coeficientes += y_puntos[i] * li.coeficients
    return coeficientes
# Definir los puntos dados
x_puntos = np.array([0,1,2, 3,4])
y_puntos = np.array([1,2,0, 2,3])
# Crear un conjunto de valores x para graficar
x = np.linspace(min(x_puntos) - 1, max(x_puntos) + 1, 400)
y = [polinomio_lagrange(xi, x_puntos, y_puntos) for xi in x]
# Graficar los puntos dados
plt.scatter(x_puntos, y_puntos, color='red', label='Puntos dados')
# Graficar el polinomio de Lagrange
plt.plot(x, y, label='Polinomio de Lagrange')
# Añadir leyenda y mostrar la gráfica
plt.legend()
plt.xlabel('x')
plt.ylabel('y')
```

```
plt.title('Interpolación de Lagrange')
plt.grid(True)
plt.show()
# Reconstruir el polinomio de Lagrange
coeficientes = reconstruccion_lagrange(x_puntos, y_puntos)
polinomio = np.poly1d(coeficientes)
# Imprimir el polinomio reconstruido
print(f"El polinomio de Lagrange es:\n{polinomio}")
```

Diferencias finitas progresivas

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h}, \quad f''(x_i) = \frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i)}{h^2}$$

Diferencias divididas Regresivas

$$f'(x_i) = \frac{f(x_i) - f(x_{i-1})}{h}, \quad f''(x_i) = \frac{f(x_i) - 2f(x_{i-1}) + f(x_{i-2})}{h^2}$$

Diferencias finitas Centrales

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1}))}{2h}, \quad f''(x_i) = \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{h^2}$$

En este gráfico se resume los tres casos posibles

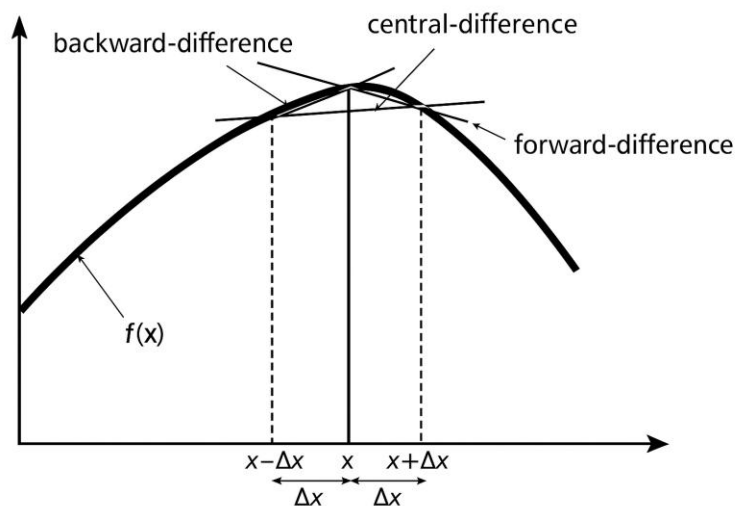


FIGURA 12

Código

```
# Definir la funcion
def f(x):
    return x**3 - 2*x + 1
# Punto y paso
x = 2
h = 0.1

# Diferencias finitas centradas para la primera derivada
def primera_derivada(f, x, h):
    return (f(x + h) - f(x - h)) / (2 * h)

# Diferencias finitas centradas para la segunda derivada
def segunda_derivada(f, x, h):
    return (f(x + h) - 2*f(x) + f(x - h)) / (h**2)

# Calcular las derivadas
primera = primera_derivada(f, x, h)
segunda = segunda_derivada(f, x, h)

# Imprimir los resultados
print(f"Primera derivada en x = {x}: {primera}")
print(f"Segunda derivada en x = {x}: {segunda}")
```

Ejercicios polinomios interpolantes y derivación numérica

Para los siguientes construir un polinomio interpolante de Lagrange, para los casos donde haya una función a la que comparar, calcule los errores globales y locales, y grafique la función y su polinomio interpolante:

1. Hallar el polinomio que pasa por los puntos (1,1), (2,4), (3,9)
2. Reconstruir la función que pasa por los puntos (0,1), (1,3), (2,2), (3,5)
3. Dado el siguiente el siguiente conjunto de puntos, hallar el valor de (b) : $x = [0,1,2,3,4]$, $y = [1,2,b,2,3]$
4. Hallar el polinomio interpolante de Lagrange para: $x = [0,1,2,3,4]$, $f(x) = [1,2,0,2,3]$
5. Hallar el polinomio interpolante de Lagrange para: $[0,1,2]$, $y = [1,3,0]$
6. Hallar el polinomio de segundo grado que pase por: $x = [1,2,3]$, $y = [10,15,80]$
7. Construir un polinomio con los datos: $x = [2,4,5]$, $f(x) = [5,6,3]$
8. Construir un polinomio que interpole: $x = [-2,0,2]$, $f(x) = [0,1,0]$
9. Aproximar $f(x) = \sin(x)$, con $x \in [0, \pi]$, use un polinomio de Lagrange de 2 grado
10. Construir el polinomio interpolante de Lagrange para: $x = [0,1,2]$, $f(x) = [1,2,7]$
11. Usar los nodos $x_0 = 2$, $x_1 = 2.5$, $x_2 = 4.5$, para construir el polinomio interpolante de Lagrange que aproxime $f(x) = \frac{1}{x}$

12. Sea $f(x) = 2\sin\left(\frac{\pi x}{6}\right)$, use los nodos $x_0 = 1, x_1 = 2, x_2 = 3$, use interpolación de Lagrange de grado 2 para aproximar:

- a) $f(4)$
- b) $f(1.5)$

13. Use los siguientes nodos $x_0 = 0, x_1 = 0.6, x_2 = 0.9$, construir que aproxime $f(0.45)$

- a) $f(x) = \cos(x)$
- b) $f(x) = \sqrt{x+1}$
- c) $f(x) = \ln(x+1)$

Ejercicios de Diferencia Finitas

- Use diferencias finitas centrales para hallar la derivada aproximada de $f(x) = \sin(x)$, en cada punto de $x = [0, 0.1, 0.2, 0.3, 0.4, 0.5]$, con $h = 0.1$
- Hallar la derivada de $f(x) = e^x$ en cada punto $x = [0, 0.1, 0.2, 0.3, 0.4, 0.5]$, con $h = 0.1$
- Sea $f(x) = x^3 - x$, calcular la derivada primera y segunda en $x = 1$, con $h = 0.1$
- Sea $f(x) = e^x \sin(x)$,
 - Halle $f'(1)$ usando diferencias finitas centrales y un paso de $h = 0.01$
 - Halle el error absoluto
 - Halle $f''(1)$ usando diferencia finitas centrales
- Comparar la aproximación por diferencias finitas de segundo orden exactas hacia adelante, atrás y centrales de $f(x) = e^{-2x} - x$, para $x = 2$

6 calcule la velocidad y la aceleración en cada punto usando el método de diferencias finitas centrales salvo en los extremos donde pueda usar progresiva o regresiva. Completar la tabla de valores

t(seg)	0	1	2	3	4	5	6	7	8
$x(m)$	0	1.9	4.2	7.8	12	17	25	32	42
$v(m/s)$									
$a(m/s^2)$									

TABLA 2

7. Igual al anterior

t(seg)	0	2	4	6	8	10	12	14	16
$x(m)$	0	0.7	2	3.5	5.4	6.5	7.5	8.5	8.5
$v(m/s)$									
$a(m/s^2)$									

TABLA 3

Analice el comportamiento de la velocidad y la aceleración

Reglas de Newton Cotes

Las reglas de Newton-Cotes son una familia de métodos de integración numérica utilizadas para aproximar integrales definidas. Se basan en la interpolación de la función a integrar mediante polinomios y en la evaluación de la integral de estos polinomios.

Regla del rectángulo medio compuesta

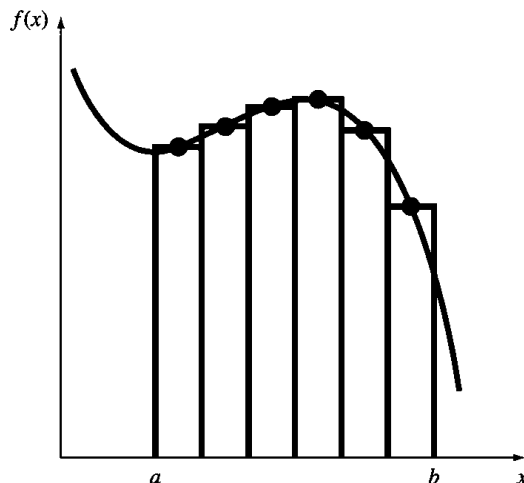


FIGURA 13

$$\int_a^b f(x) dx \approx h \sum_{i=1}^n f\left(\frac{x_{i-1} + x_i}{2}\right)$$

$$\int_a^b f(x) dx \approx h \sum_{i=0}^{n-1} f(\bar{x}_i)$$

Donde $\bar{x}_i = \text{punto medio}$, $h = \frac{(b-a)}{n}$

Existen varias reglas de Newton-Cotes, dependiendo del número de puntos utilizados en la interpolación.

Regla del Trapecio

Regla del trapecio (Newton-Cotes de orden 1): Utiliza dos puntos para aproximar la integral como la suma de áreas de un trapecioide.

$$\int_a^b f(x) dx \approx \frac{(b-a)}{2} [f(a) + f(b)]$$

Forma compuesta

Se divide el área de subintervalos para mejorar la precisión, acá se resume la formula:

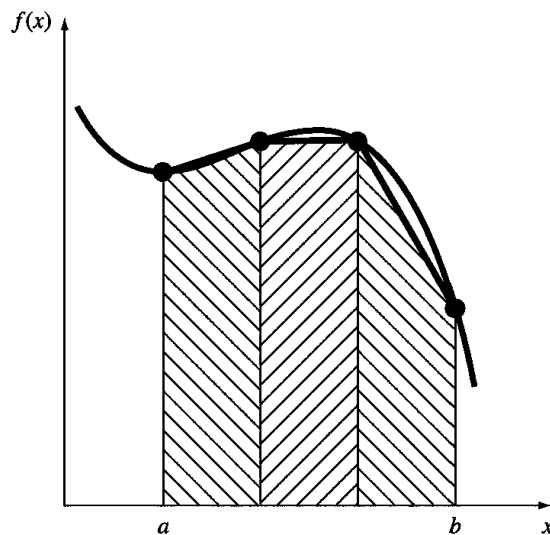


FIGURA 14

$$\int_a^b f(x)dx \approx \frac{h}{2} \left(f(a) + 2 \sum_{i=1}^{n-1} f(a + ih) + f(b) \right)$$

Donde h es el paso, y n el número de subintervalos:

$$h = \frac{(b - a)}{n}$$

Error de truncamiento

$$E_T = -\frac{(b - a)^3}{12n^2} f''(\xi)$$

Regla de Simpson

Regla de Simpson 1/3, (Newton-Cotes)

a) Usa tres puntos y ajusta un polinomio cuadrático para mejorar la precisión. Para esta regla (n) solo puede ser par

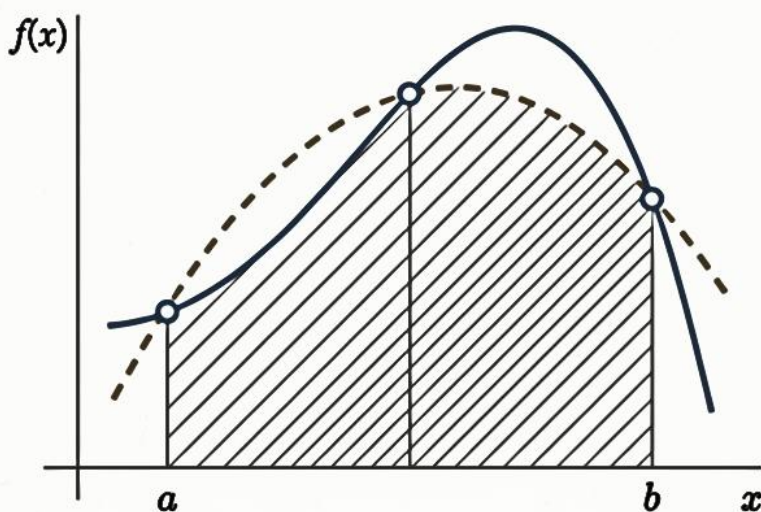


Figura 15

$$\int_a^b f(x) dx \approx \frac{h}{3} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right)$$

$$h = \frac{(b-a)}{2}$$

Error de truncamiento

$$E = -\frac{(b-a)^5}{2880} f^{(4)}(\xi)$$

donde $\xi \in [a, b]$

$$E = -\frac{h^5}{90} f^{(4)}(\xi)$$

Reglas de Simpson 1/3 compuesta

$$\int_a^b f(x) dx \approx \frac{h}{3} \left(f(a) + 4 \sum_{\text{impares}} f(a+ih) + 2 \sum_{\text{pares}} f(a+ih) + f(b) \right)$$

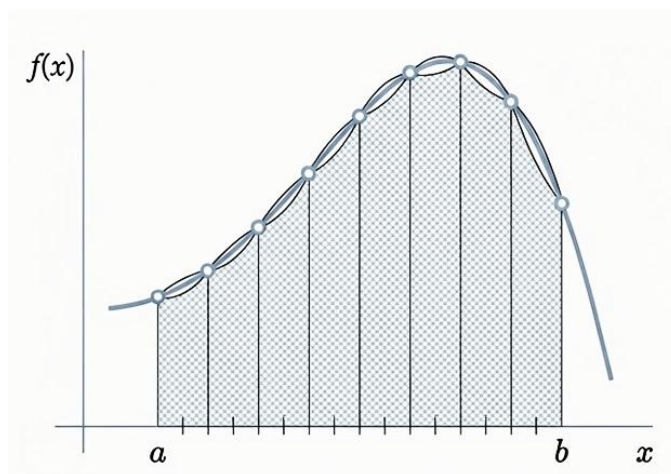


FIGURA 16

Error de truncamiento

$$E_{comp} = -\frac{(b-a)^5}{180n^4} f^{(4)}(\xi)$$

$$E_{comp} = -\frac{(b-a)}{180} h^4 f^{(4)}(\xi)$$

$$\begin{cases} h = \frac{(b-a)}{n} \\ n \text{ es par} \end{cases}$$

Simpson (3/8)

Reglas de Simpson 3/8 y otras de orden superior: Utilizan más puntos y polinomios de mayor grado para obtener aproximaciones más precisas. Es una extensión de la regla 1/3 pero esta usa 4 puntos. $n=3$

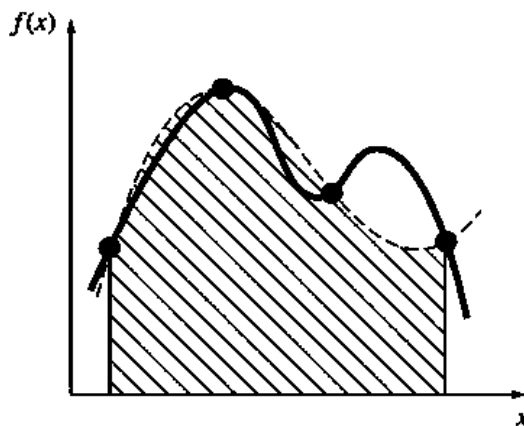


FIGURA 17

$$\int_a^b f(x)dx \approx \frac{3h}{8} (f_{(a)} + 3f(x_1) + 3f(x_2) + f_{(b)})$$

Error de truncamiento

$$h = \frac{(b-a)}{3}$$

$$E_T = -\frac{3}{80} h^5 f^{(4)}(\xi)$$

Simpson 3/8 compuesta, n debe ser múltiplo de 3

$$\int_a^b f(x)dx \approx \frac{3h}{8} \left(f_{(a)} + 3 \sum_{\text{impares}} f(x_i) + 3 \sum_{\text{pares}} f(x_j) + 2 \sum_{\text{mult. 3}} f(x_i) + f_{(b)} \right)$$

$$h = \frac{(b-a)}{3n}$$

Error de truncamiento

$$E = -\frac{(b-a)^5}{6480} f^{(4)}(\xi)$$

Codigo1: regla del rectángulo medio compuesta

```
import numpy as np
# Definimos la función a integrar
def funcion(x):
    return np.sin(x) # Puedes cambiarla por otra función
# Límites de integración
a = 0 # Límite inferior
b = np.pi # Límite superior
# Número de subintervalos
n = 10 # Ajusta según la precisión deseada
# Paso
h = (b - a) / n
# Puntos medios
x_medio = np.linspace(a + h/2, b - h/2, n)
# Aplicamos la regla del rectángulo medio compuesta
integral = h * np.sum(funcion(x_medio))
print(f"Integral aproximada con la regla del rectángulo medio compuesta: {integral:.4f}")
```

codigo2: regla del trapecio simple

```
import numpy as np
# Definimos la función a integrar
def funcion(x):
    return np.sin(x) # Puedes cambiarla por cualquier otra función
# Límites de integración
a = 0 # Límite inferior
b = np.pi/2 # Límite superior
# Evaluamos la función en los extremos
fa = funcion(a)
fb = funcion(b)
# Aplicamos la regla del trapecio simple
integral = (b - a) / 2 * (fa + fb)
print(f"Integral aproximada con la regla del trapecio simple: {integral:.4f}")
```

codigo 3: regla del trapecio compuesta

```
import numpy as np
# Definimos la función a integrar
def funcion(x):
    return np.sin(x) # Puedes cambiarla por otra función
# Límites de integración
a = 0 # Límite inferior
b = np.pi # Límite superior
# Número de subintervalos
n = 10 # Ajusta según la precisión deseada
# Paso
h = (b - a) / n
# Puntos de evaluación
x = np.linspace(a, b, n + 1)
y = funcion(x)
# Aplicamos la regla del trapecio compuesta
integral = (h / 2) * (y[0] + 2 * np.sum(y[1:n]) + y[-1])
print(f"Integral aproximada con la regla del trapecio compuesta: {integral:.4f}")
```

codigo4: regla de Simpson simple

```
import numpy as np
# Definimos la función a integrar
def funcion(x):
    return np.sin(x) # Puedes cambiarla por cualquier otra función
# Límites de integración
a = 0 # Límite inferior
b = np.pi # Límite superior
# Punto medio
```

```
m = (a + b) / 2
# Evaluamos la función en los tres puntos
fa = funcion(a)
fm = funcion(m)
fb = funcion(b)
# Aplicamos la regla de Simpson simple
integral = (b - a) / 6 * (fa + 4 * fm + fb)
print(f"Integral aproximada con Simpson simple: {integral:.4f}")
```

codigo 5: regla de Simpson compuesta

```
import numpy as np
# Definimos la función a integrar
def funcion(x):
    return np.sin(x) # Puedes cambiarla por cualquier otra función
# Límites de integración
a = 0 # Límite inferior
b = np.pi # Límite superior
# Número de subintervalos (debe ser par)
n = 10 # Ajusta según la precisión deseada
# Paso
h = (b - a) / n
# Puntos
x = np.linspace(a, b, n + 1)
y = funcion(x)
# Aplicamos la regla de Simpson compuesta
S = y[0] + y[-1] + 4 * np.sum(y[1:n:2]) + 2 * np.sum(y[2:n-1:2])
integral = (h / 3) * S
print(f"Integral aproximada con Simpson compuesta: {integral:.4f}")
```

codigo 6: regla de Simpson 3/8 simple

```
import numpy as np
# Definimos la función a integrar
def funcion(x):
    return np.sin(x) # Puedes cambiarla por otra función
# Límites de integración
a = 0 # Límite inferior
b = np.pi # Límite superior
# Paso
h = (b - a) / 3
# Puntos de evaluación
x1 = a + h
x2 = a + 2*h
# Evaluamos la función en los puntos
fa = funcion(a)
```

```

fx1 = funcion(x1)
fx2 = funcion(x2)
fb = funcion(b)
# Aplicamos la regla de Simpson 3/8 simple
integral = (3 * h / 8) * (fa + 3 * fx1 + 3 * fx2 + fb)
print(f"Integral aproximada con Simpson 3/8 simple: {integral:.4f}")

```

codigo 7: la regla de Simpson 3/8 compuesta

```

import numpy as np
# Definimos la función a integrar
def funcion(x):
    return np.sin(x) # Puedes cambiarla por otra función
# Límites de integración
a = 0 # Límite inferior
b = np.pi # Límite superior
# Número de subintervalos (debe ser múltiplo de 3)
n = 9 # Ajusta según la precisión deseada
# Paso
h = (b - a) / n
# Puntos de evaluación
x = np.linspace(a, b, n + 1)
y = funcion(x)
# Aplicamos la regla de Simpson 3/8 compuesta
S = y[0] + y[-1] + 3 * np.sum(y[1:n:3]) + 3 * np.sum(y[2:n:3]) + 2 * np.sum(y[3:n-1:3])
integral = (3 * h / 8) * S
print(f"Integral aproximada con la regla de Simpson 3/8: {integral:.4f}")

```

Ejercicios

1. Sea $\int_0^{\pi/2} (6 + 3 \cos(x)) dx$:

- Use la regla del trapecio para aproximar la solución con $n = 2, n = 4$
- Use la regla de Simpson (1/3) con $n = 4$
- Use la regla de Simpson (3/8) con $n = 3, y n = 6$

2. Sea $\int_1^2 \left(x + \frac{2}{x}\right)^3 dx$

- Use la solución analítica para calcular los errores relativos porcentuales para la regla del trapacio.

3. Sea $\int_{-3}^3 (4x - 3)^3 dx$

a) Integre de forma analítica y use la regla de Simpson con $n = 3$, y $n = 6$ para comparar la aproximación (use 3/8)

4. Sea $\int_0^3 (x^2 e^x) dx$:

a) Integre de forma analítica y numérica, emplee la regla del trapecio con $n = 4$

b) Use la regla de Simpson (1/3) con $n = 4$

c) Calcule los errores de truncamiento

5. Sea $\int_0^4 (1 - e^{-2x}) dx$:

a) Integre con la regla del trapecio con $n = 4$

b) Integre usando la regla de Simpson (1/3) con $n = 4$

6. Sea $\int_0^\pi (\sin(x)) dx$

a) Integre con la regla del trapecio con $n = 10$

b) Integre usando la regla de Simpson (1/3) con $n = 4$

7. Sea $\int_0^1 e^{x^2} dx$

a) Integre con la regla de Simpson (1/3) con, $n = 4$, $n = 10$

b) Integre usando la regla del rectángulo con $n = 5$

8. Sea $\int_0^2 x^2 dx$

a) Integre con la regla del rectángulo punto medio con, $n = 4$

9. Sea $\int_0^2 \sqrt[4]{1+x^2} dx$

a) Integre usando la regla del rectángulo izquierdo con, $n = 8$

10. Sea $\int_1^6 \frac{x^2}{6} dx$

a) Integre usando la regla del trapecio con, $n = 5$

11. Sea $\int_{-1}^1 e^{x^4} dx$

a) Integre usando la regla del trapecio con, $n = 5$

12. Sea $\int_1^2 \frac{1}{x^2} dx$

a) Integre usando la regla del trapecio con, $n = 5$

El método de Monte Carlo

El método de Monte Carlo es una técnica matemática utilizada para aproximar soluciones numéricas a problemas complejos mediante el uso de muestreo aleatorio y probabilidad. Su nombre proviene del famoso casino de Monte Carlo, debido al enorme papel del azar en este método.

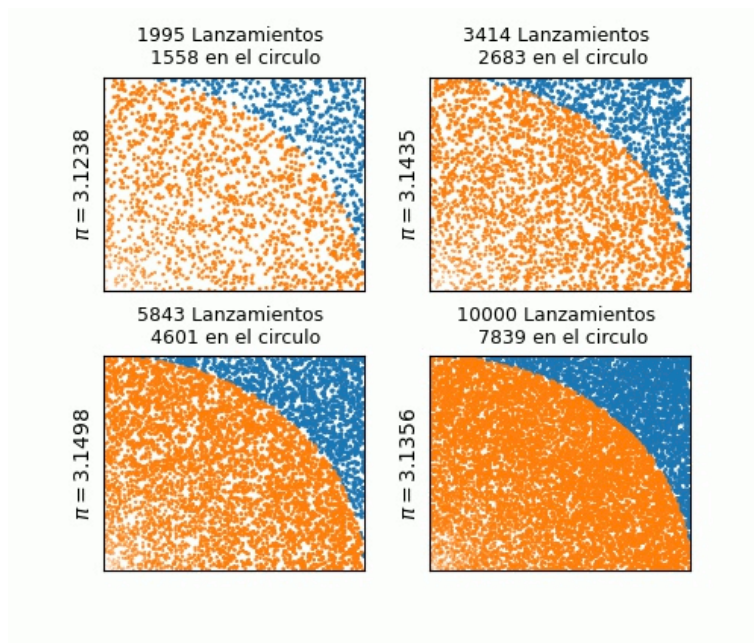


FIGURA 18

Cómo funciona

El método de Montecarlo utiliza simulaciones repetitivas basadas en números aleatorios para aproximar soluciones a problemas complejos. Se suele emplear en situaciones donde es difícil o imposible obtener una solución analítica exacta. Su procedimiento básico sigue estos pasos:

1. Definir el problema y el dominio de posibles valores.
2. Generar valores aleatorios dentro del dominio.
3. Evaluar cada valor en la función del problema.
4. Calcular la estimación basada en los resultados obtenidos.

Características Principales

1. Se basa en repeticiones aleatorias (simulaciones) para estimar resultados.
2. Es útil cuando el problema es demasiado complicado para resolverse analíticamente.
3. Se aplica en áreas como física, finanzas, ingeniería, inteligencia artificial y mas.

Método gráfico para integración numérica

En estos casos se busca inscribir el área de interés en un cuadrado o rectángulo, generamos numeros aleatorios de tan manera que se pueda comparar los numeros que quedan bajo la curva que se llaman puntos de exitos con el total de los puntos generados.

Supongamos que quieres estimar el área de una región R en el plano. La idea es:

1. Encerrar la región R en una caja rectangular que sea fácil de describir. Por ejemplo, que tenga límites $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$, $[y_{\{min\}}, y_{\{max\}}]$.
2. Generar muchos puntos aleatorios uniformemente distribuidos dentro del rectángulo.
3. Contar qué fracción de puntos cae dentro de R.
4. Calcular el área estimada como:

$$\text{Área}(R) \approx \text{Área rectángulo} \times \frac{\text{número de puntos dentro de } R}{\text{número total de puntos}}$$

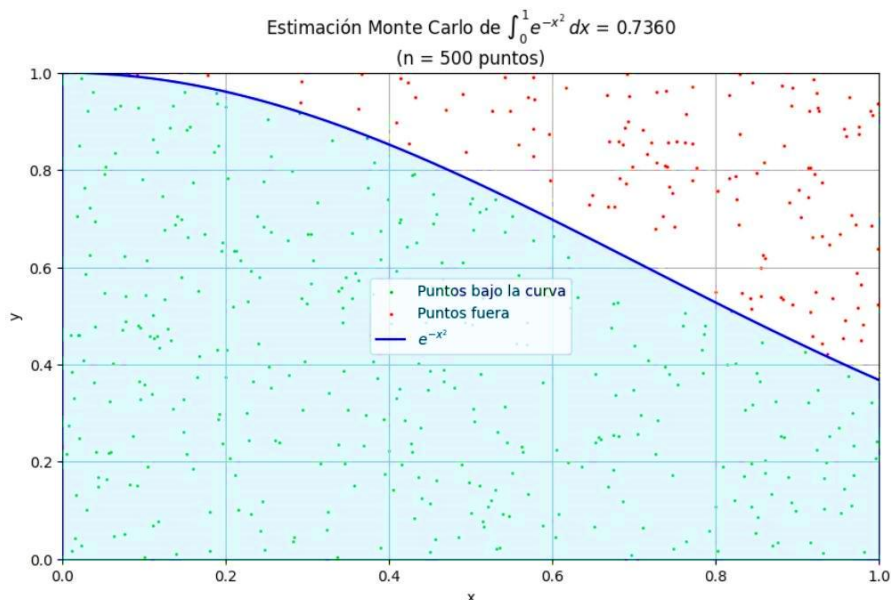


FIGURA 19

Este ejemplo se estimo con el uso de 500 puntos aleatorios, basicamente usando un código que los genere y cuente cuando quedan debajo de la curva y con esa informacion establecemos la relación, ahora si queremos más precisión es necesario aumente el número de muestras generadas.

Ejemplo clásico estimar π (pi).

El objetivo es estimar pi usando aleatoriedad.

Algunos pasos:

1. Dibuja un cuadrado de lado 2 y un circulo inscrito de radio 1 (área del circulo = π).

2. Genera puntos aleatorios dentro del cuadrado.
3. Cuenta los puntos que caen dentro del círculo (puntos de éxito), la distancia al centro ≤ 1

$$\pi \approx 4 \left(\frac{\text{puntos dentro del círculo}}{\text{total de puntos}} \right)$$

Código python

```
import random
# Número de puntos aleatorios
num_puntos = 1000000
puntos_dentro = 0 # Contador de puntos dentro del círculo
# Generamos los puntos aleatorios
for _ in range(num_puntos):
    x = random.uniform(-1, 1) # Coordenada x aleatoria en [-1,1]
    y = random.uniform(-1, 1) # Coordenada y aleatoria en [-1,1]

    # Si el punto cae dentro del círculo unitario, sumamos al contador
    if x**2 + y**2 <= 1:
        puntos_dentro += 1
# Estimación de pi usando la relación entre área del círculo y cuadrado
pi_estimado = (puntos_dentro / num_puntos) * 4
print(f"Estimación de π usando Montecarlo: {pi_estimado}")
```

Herramientas estadísticas necesarias (Distribucion Normal)

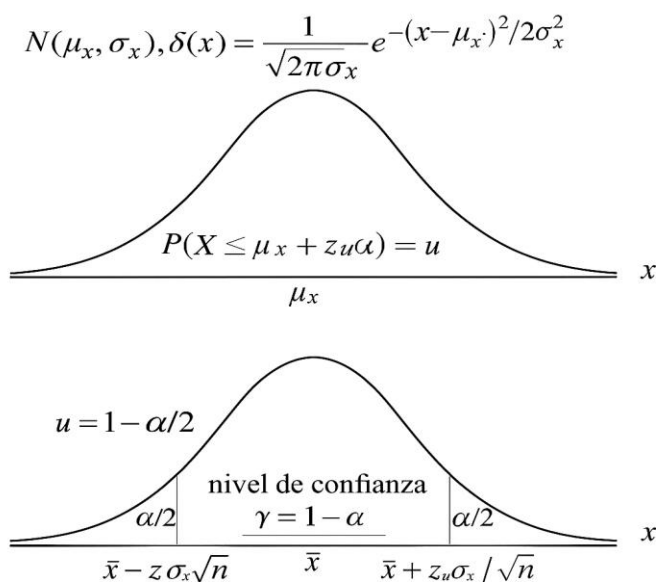


FIGURA 20

Formulas:

$$\sigma = \sqrt{\frac{1}{(n-1)} \sum (f(x_i) - \bar{x})^2}$$

$\sigma =$ desviación estándar

$$EE = \frac{\sigma}{\sqrt{n}}$$

$EE =$ Error estándar

$$\left[\hat{I} - z_{\alpha/2} \frac{\sigma}{\sqrt{n}}, \hat{I} + z_{\alpha/2} \frac{\sigma}{\sqrt{n}} \right]$$

(Intervalo de confianza)

$\hat{I} =$ Integral estimada

$\hat{I} \pm z_{\alpha/2} \frac{\sigma}{\sqrt{n}}$ (para calcular los límites)

$$\hat{I} = \int_a^b f(x) dx = (b-a) \frac{1}{n} \sum_{i=1}^n f(x_i)$$

IC	90	95	99
$z_{\alpha/2}$	1.645	1.96	2.576

TABLA 4

Métodos para generar números con distribución uniforme y los generadores más comunes en programación incluyen:

❖ en Python: `random.uniform(a,b)` Devuelve un número aleatorio en el intervalo $[a, b]$.

```
import random
# Generar 5 números aleatorios uniformes en el rango [0, 1]
for _ in range(10):
    print(random.uniform(0, 1))
```

Primera ejecución 0.5415297812161269
 0.08528259751738321
 0.978464626756584
 0.6283191030955314

Segunda ejecución 0.5415297812161269
 0.08528259751738321
 0.978464626756584
 0.6283191030955314

Los valores cambian para cada ejecución del código